Sorting

Java: Sort.java

COMP 182 · Lecture 07

How sorting algorithms work

Trace the steps. Explain the cost. Choose a method.

The sorting problem

Java: StabilityDemo.java Java: SortItem.java

Rearrange the input so a[0] <= a[1] <= ... <= a[n - 1].

Keep every original item, including repeated values.

Compare algorithms by time, extra space, and stability.

A stable sort preserves the original order of equal keys.

Bubble sort

Java: BubbleSort.java

Compare adjacent items and swap them when they are out of order.

After each left-to-right pass, the largest remaining item is final.

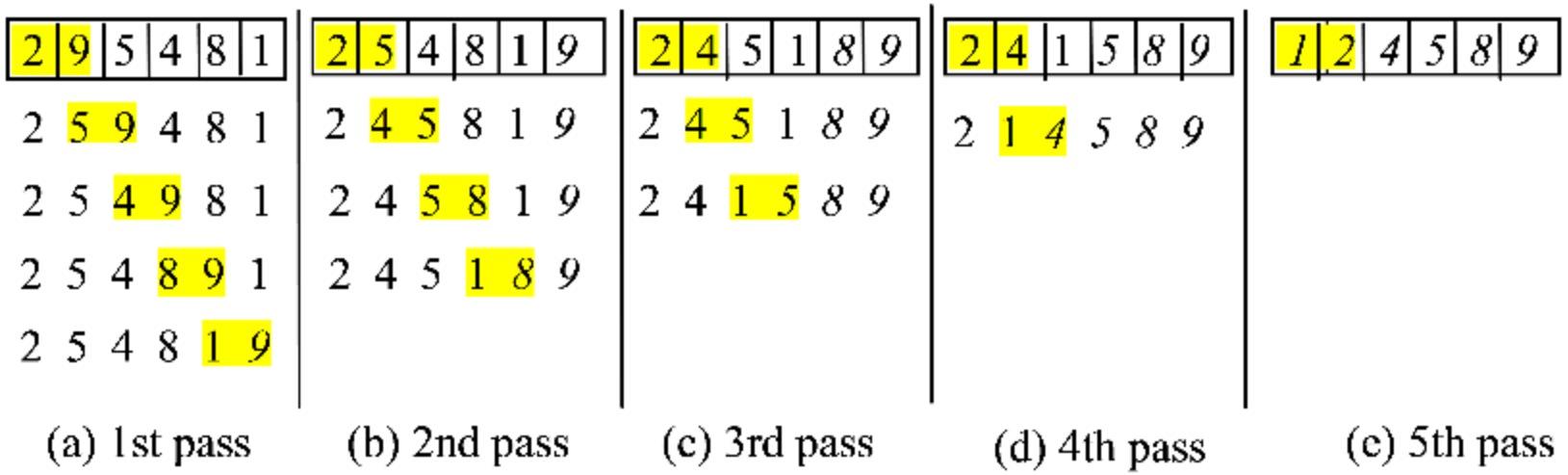
Shorten the next pass because the sorted suffix is complete.

Stop early when an entire pass makes no swaps.

Best: O(n) with early exit. Average and worst: $O(n^2)$.

Bubble sort: pass-by-pass trace

Java: BubbleSort.java



Input: 2, 9, 5, 4, 8, 1. Each pass fixes one more item at the right.

Selection sort

Java: SelectionSort.java

Java: StabilityDemo.java

```
for (int i = 0; i < a.length - 1; i++) {
  int min = i;
  for (int j = i + 1; j < a.length; j++)
    if (a[j] < a[min]) min = j;
  swap(a, i, min);
}
```

Find the minimum in the unsorted suffix, then swap it into position i.

All input orders take $O(n^2)$ time. Extra space: O(1).

Insertion sort

Java: InsertionSort.java

```
for (int i = 1; i < a.length; i++) {
  int value = a[i];
  int j = i - 1;
  while (j >= 0 && a[j] > value) {
    a[j + 1] = a[j];
    j--;
  }
  a[j + 1] = value;
}
```

Insert the next value into the sorted prefix by shifting larger values.

Best: O(n). Average and worst: O(n²). Extra space: O(1).

Insertion sort: growing the sorted prefix

Java: InsertionSort.java

Inserted value	Sorted prefix	Remaining suffix
Initial	2	9, 5, 4, 8, 1, 6
9	2, 9	5, 4, 8, 1, 6
5	2, 5, 9	4, 8, 1, 6
4	2, 4, 5, 9	8, 1, 6
8	2, 4, 5, 8, 9	1, 6
1	1, 2, 4, 5, 8, 9	6
6	1, 2, 4, 5, 6, 8, 9	Empty

Every row preserves all values and extends the sorted prefix by one.

Merge sort: divide, sort, merge

Java: MergeSort.java

Split the range into two roughly equal halves.

Recursively sort each half.

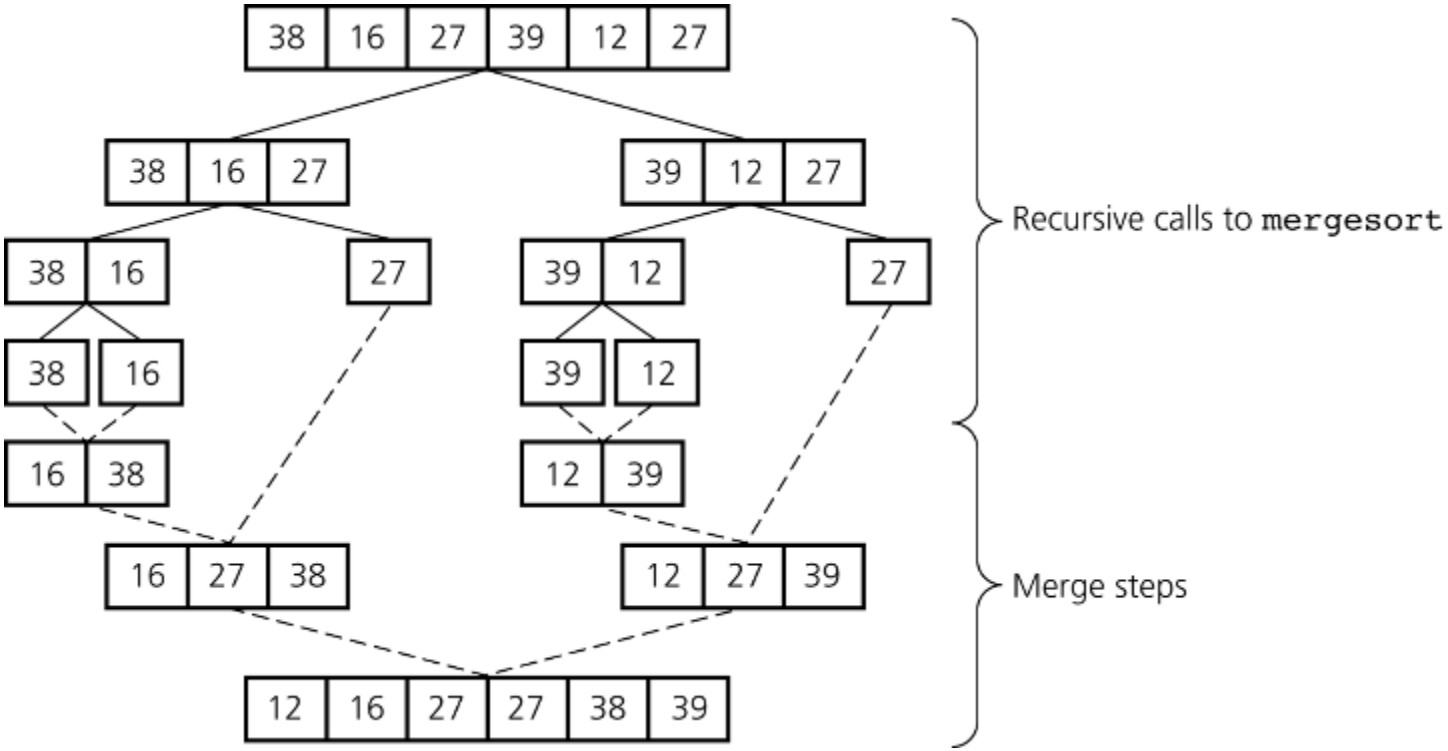
Merge the two sorted halves into one sorted range.

Base case: a range of zero or one item is already sorted.

Standard array merge sort takes O(n log n) time and O(n) extra array space.

Merge sort: a complete trace

Java: MergeSort.java



Split until each run has one item, then merge in sorted order.

Recursive merge sort

Java: MergeSort.java

```
mergeSort(a, temp, lo, hi):
  if hi - lo <= 1: return
  mid = lo + (hi - lo) / 2
  mergeSort(a, temp, lo, mid)
  mergeSort(a, temp, mid, hi)
  merge(a, temp, lo, mid, hi)
```

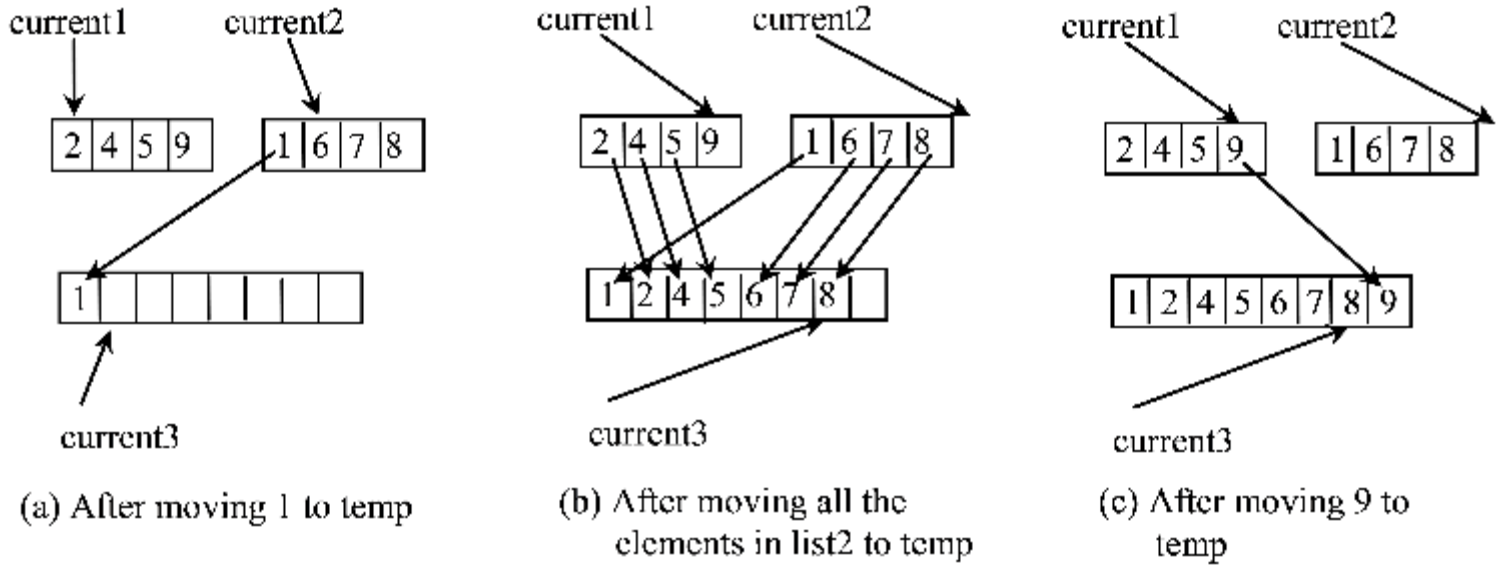
Pseudocode uses half-open ranges: lo is included and hi is excluded.

Allocate temp once and reuse it throughout the recursive calls.

Merging two sorted runs

Java: MergeRunsDemo.java

Java: MergeSort.java



Compare the first unused item from each run.

Copy the smaller item, then advance that run's cursor.

When one run ends, copy the rest of the other run.

Merge sort: work at each level

Java: MergeSort.java

Recursion level	Number of runs	Run size	Total merge work
0	1	n	O(n)
1	2	n / 2	O(n)
2	4	n / 4	O(n)
k	2^k	n / 2^k	O(n)

About log2(n) levels, with O(n) work per level, gives O(n log n).

Merge sort: time and space

Java: MergeSort.java Java: StabilityDemo.java

Standard top-down version: O(n log n) in best, average, and worst cases.

Temporary array: O(n). Recursive stack: O(log n).

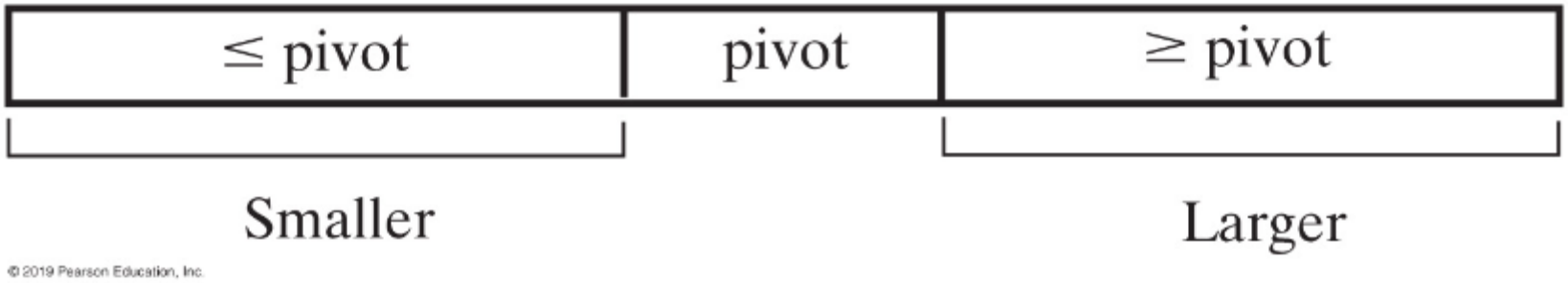
Stable when the left run wins ties during merging.

Useful when predictable time and stability matter.

Tradeoff: extra array storage in return for a worst-case time guarantee.

Quicksort: partition around a pivot

Java: QuickSort.java



© 2019 Pearson Education, Inc.

Choose a pivot and partition the current range.

In this variant, partition puts the pivot in its final position.

Recursively sort the ranges on either side.

The two parts can have very different sizes.

Recursive quicksort

Java: QuickSort.java

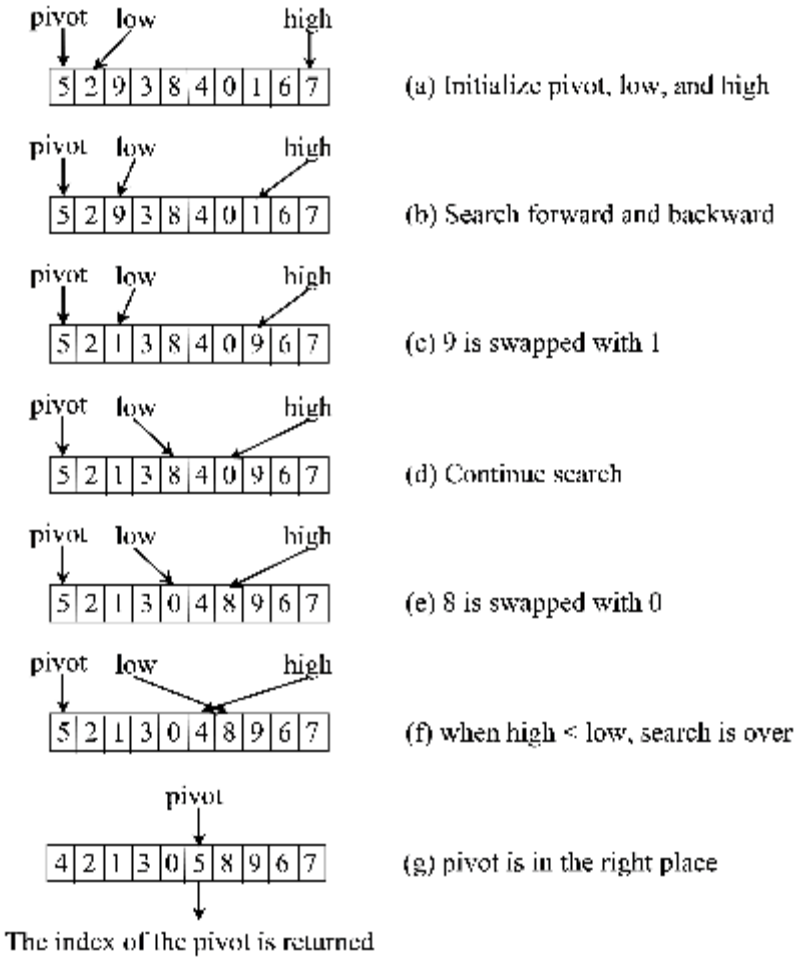
```
quickSort(a, first, last):
  if first >= last: return
  p = partition(a, first, last)
  quickSort(a, first, p - 1)
  quickSort(a, p + 1, last)
```

Pseudocode uses inclusive bounds: both first and last belong to the range.

The pivot is final, so both recursive calls exclude its position.

Quicksort: partition trace

Java: QuickSort.java



Pivot = 5. The two cursors find misplaced values, then the pivot moves into place.

Two-cursor partition

Java: QuickSort.java

```
partition(a, first, last):
  pivot = a[first]
  low = first + 1; high = last
  while low <= high:
    while low <= high and a[low] <= pivot:
      low++
    while low <= high and a[high] > pivot:
      high--
    if low < high: swap(a, low, high)
  swap(a, first, high)
  return high
```

The cursors only move inward.

One partition takes O(n) time and O(1) extra space.

Quicksort: balanced and unbalanced splits

Java: QuickSort.java

Split pattern	Recursion depth	Total time
Roughly equal parts	O(log n)	O(n log n)
One item, then n - 1	O(n)	O(n^2)

With distinct keys, randomized pivots give expected O(n log n). Worst case: O(n^2).

Partitioning one range is linear.

Standard recursion uses expected O(log n) stack space and O(n) in the worst case.

Comparison sorts and integer keys

Java: BucketSort.java Java: RadixSort.java

Comparison sorts learn order by comparing pairs of keys.

Worst-case comparison sorting needs $\Omega(n \log n)$ comparisons.

Bucket and radix methods use the structure of the keys.

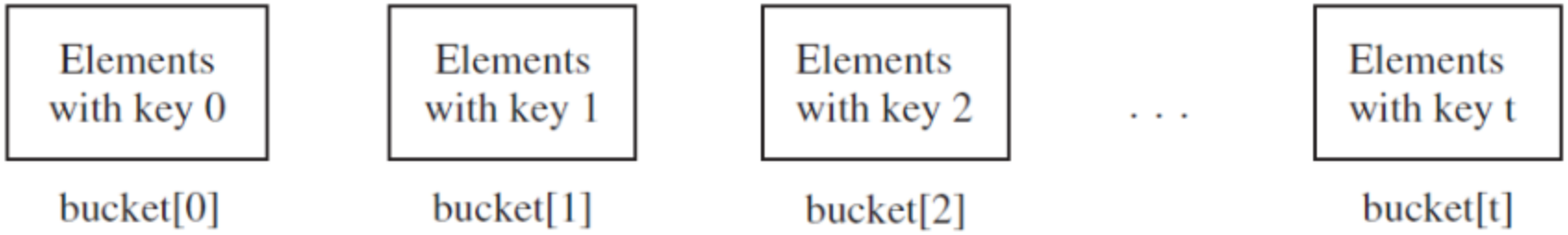
Their costs depend on the key range, number of digits, and radix.

The lower bound concerns general comparison sorting in the worst case.

Direct-address bucket sort

Java: BucketSort.java

Java: SortItem.java



Assume integer keys in the range 0 through K - 1.

Append each item to the bucket that matches its key.

Read buckets in increasing key order. Preserve order within each bucket.

Time: O(n + K). Extra storage: O(n + K) for lists of records.

LSD radix sort: a complete example

Java: RadixSort.java

Pass	Sequence after stable distribution and collection
Input	331, 454, 230, 34, 343, 45, 59, 453, 345, 231, 9
Ones	230, 331, 231, 343, 453, 454, 34, 45, 345, 59, 9
Tens	9, 230, 331, 231, 34, 343, 45, 345, 453, 454, 59
Hundreds	9, 34, 45, 59, 230, 231, 331, 343, 345, 453, 454

Treat 9 as 009 and 34 as 034. Every digit pass must be stable.

Radix sort: stable digit passes

Java: RadixSort.java

```
for digit = least to most significant:
  create b empty buckets
  for each item in its current order:
    append item to bucket[digitValue(item)]
  collect buckets in order 0 through b - 1
```

d digit passes, n items, b possible digit values.

Time: O(d(n + b)). Extra storage: O(n + b).

Signed values and variable-length keys need explicit handling.

Sorting algorithm comparison

Java: Sort.java Java: TreeSort.java Java: HeapSort.java

Algorithm	Average time	Worst time	Extra space
Selection	$O(n^2)$	$O(n^2)$	O(1)
Bubble, early exit	$O(n^2)$	$O(n^2)$	O(1)
Insertion	$O(n^2)$	$O(n^2)$	O(1)
Merge, array	O(n log n)	O(n log n)	O(n)
Quick, standard	O(n log n)	$O(n^2)$	O(log n)*
LSD radix	O(d(n + b))	O(d(n + b))	O(n + b)
Tree, unbalanced BST	O(n log n)	$O(n^2)$	O(n)
Heap	O(n log n)	O(n log n)	O(1)

Quick stack: expected O(log n), worst O(n). Quick/tree averages assume random distinct keys.